



**SP.TOP: DATA SCIENCE AND ANALYTICS
COMP4381**

**Project Fourth Phase
Flight Delay Prediction Using Machine Learning**

Prepared by:

Yaqeen Azamta	1221736
Ramz Abufarha	1230963
Anas Hamdan Al-Haj	1230038

Dr. Ahmad Sabbah
Section 2
27/6/2026

Abstract

Flight delays affect both passengers and airlines by disrupting schedules and increasing operational costs. In this project, we investigated whether machine-learning models could predict flight delays using data collected from five major European cities: London, Amsterdam, Paris, Frankfurt, and Barcelona. The dataset combined public flight records with airport information and historical weather data.

The prediction task was formulated as a binary classification problem, where the goal was to determine whether a flight would be delayed by more than fifteen minutes. Since only a small percentage of flights met this condition, we evaluated the models using precision, recall, and F1 score instead of relying only on accuracy. Six machine-learning algorithms were compared, and Random Forest achieved the best overall performance, reaching an F1 score of approximately 0.31 after optimization.

Our results showed that operational features, such as previous delays and route history, contributed more to the predictions than weather-related features. Although several optimization techniques were tested, all models achieved similar performance levels. This suggests that the main limitation is the available data rather than the machine-learning algorithms themselves, as many important operational factors are not included in public flight datasets.

Table of Contents

Contents

Abstract	2
Table of Contents	3
Table of Figures	5
Table of Tables	5
1. Introduction.....	6
2. Dataset Description	Error! Bookmark not defined.
Dataset Characteristics.....	Error! Bookmark not defined.
Selected Cities	Error! Bookmark not defined.
Number of Flights by City	Error! Bookmark not defined.
3. Data Sources.....	Error! Bookmark not defined.
4. Data Preparation	Error! Bookmark not defined.
Data Cleaning	Error! Bookmark not defined.
Feature Engineering	Error! Bookmark not defined.
Delay Categories.....	Error! Bookmark not defined.
5. Weather Data Integration	Error! Bookmark not defined.
6. Exploratory Data Analysis.....	Error! Bookmark not defined.
Delay Statistics.....	Error! Bookmark not defined.
Interpretation	Error! Bookmark not defined.
Delayed Flights (>15 Minutes).....	Error! Bookmark not defined.
Interpretation	Error! Bookmark not defined.
Delay Categories Distribution.....	Error! Bookmark not defined.
Interpretation	Error! Bookmark not defined.
7. Weather Impact Analysis.....	Error! Bookmark not defined.
Average Delay by Weather Condition	Error! Bookmark not defined.
Interpretation	Error! Bookmark not defined.

Bad Weather vs Normal Weather	Error! Bookmark not defined.
Interpretation	Error! Bookmark not defined.
8. Seasonal Analysis.....	Error! Bookmark not defined.
Interpretation	Error! Bookmark not defined.
9. Correlation Analysis.....	Error! Bookmark not defined.
Interpretation	Error! Bookmark not defined.
10. Aggregation Effect Discussion	Error! Bookmark not defined.
11. Findings	18
12. Limitations	19
13. Conclusion	20
References.....	21

Table of Figures

Figure 1 : Number of Flights by Origin CityError! Bookmark not defined.

Figure 2 Distribution of Flight Delay MinutesError! Bookmark not defined.

Figure 3 Number of Flights by Delay CategoryError! Bookmark not defined.

Figure 4 Average Delay by Weather Condition GroupError! Bookmark not defined.

Figure 5 Average Delay: Normal Weather vs Bad Weather.....Error! Bookmark not defined.

Figure 6 Correlation Matrix of Numerical FeaturesError! Bookmark not defined.

Table of Tables

Table 1 Selected Cities Included in the Dataset Error! Bookmark not defined.

Table 2 Number of Flights by Origin City..... Error! Bookmark not defined.

Table 3 Summary Statistics of Flight Delays Error! Bookmark not defined.

Table 4 Percentage of Delayed and Non-Delayed Flights Error! Bookmark not defined.

Table 5 Flight Counts by Delay Category Error! Bookmark not defined.

Table 6 Delay Statistics by Weather Condition Group Error! Bookmark not defined.

Table 7 Average Delay by Season and Weather Type Error! Bookmark not defined.

1. Introduction

Flight delays can happen for many different reasons, including weather conditions, airport congestion, air traffic control restrictions, technical problems, and delays that carry over from earlier flights. Since these factors often affect one another, predicting flight delays is not an easy task. However, being able to predict delays can help airlines and airports improve their planning while also allowing passengers to make better travel decisions.

In the previous phases of this project, we defined the problem, reviewed related work, and explored a flight dataset enriched with airport and weather information. In this phase, our focus is on developing and evaluating machine-learning models that predict whether a flight will be delayed by more than fifteen minutes. Several models are compared, optimized, and evaluated to determine which approach performs best.

Rather than reporting model scores alone, this phase also explains the reasoning behind the modeling decisions, discusses the strengths and weaknesses of each approach, and analyzes why the models achieved their final performance. This provides a better understanding of both the prediction problem and the limitations of the available flight data.

2. Dataset and Problem Framing

The analytical dataset for this phase was rebuilt directly from the raw source so that the operational features described later could be computed correctly. Three sources were combined: the public “Flight Delays Europe 2023–2025” flight records on Hugging Face, airport metadata from OurAirports, and daily historical weather from the Open-Meteo archive API. After filtering to the five target cities and merging the supporting data, the dataset used for this phase contains 2,081,394 flight records described by 65 columns, with origin city, airport coordinates, scheduled flight characteristics, engineered time and traffic features, weather, and operational history.

Each row represents a single flight departing from one of the five selected cities. The proportion of flights delayed by more than fifteen minutes is 4.99 percent, confirming that delayed flights are a small minority of all departures.

Table 1: Cities and countries included in the analytical dataset

City	Country
London	United Kingdom
Amsterdam	Netherlands
Paris	France
Frankfurt	Germany
Barcelona	Spain

2.1 Why Classification Instead of Regression

The second-phase plan proposed a regression model that would predict the exact number of delay minutes. The exploratory analysis in the previous phase showed that this target is extremely difficult to model: the median delay is essentially zero (about 0.08 minutes), the large majority of flights fall within a few minutes of schedule, and the distribution has a long tail of rare extreme values reaching beyond eight hundred minutes. A target dominated by unobserved operational causes and shaped by rare outliers offers very little signal for predicting an exact duration.

For this reason the problem was reframed as predicting delay occurrence, that is, whether a flight is delayed by more than fifteen minutes (the column `delayed_15min`, where 1 means delayed and 0 means not). This is a question the available features can support far more reliably than exact-minute prediction. The change should be read as letting the data guide the modeling choice rather than as a contradiction of the earlier plan.

2.2 Why F1 Rather Than Accuracy

Because only 4.99 percent of flights are delayed, a trivial model that always predicts “not delayed” would achieve about 95 percent accuracy while catching none of the delays it is supposed to find. Accuracy is therefore a misleading metric for this problem. The project instead evaluates precision, recall, and F1 on the delayed (minority) class, which directly measure how well the model identifies the delays that matter. This choice is stated here because it governs every result that follows.

3. Data Preprocessing

Several cleaning and integration steps were applied before any feature was engineered. Each step is described together with the reason it was needed.

- **Combining three sources.** No single source contained operations, geography, and weather together, so the flight records were joined to airport metadata by airport code (adding city, country, coordinates, elevation, and airport type) and to daily weather drawn from each departure airport's coordinates.
- **Duplicate removal.** Exact duplicate flight records were removed so that repeated rows would not bias frequency counts or model training.
- **Missing-value handling.** Records with no airport geography were dropped, because without a city, country, or coordinates a row cannot support regional or weather features. Sparse text fields such as aircraft model and operator were filled with “Unknown” so that otherwise-useful rows were retained. Missing weather values were filled with the median of the corresponding column.
- **Type conversion.** Date and timestamp fields were converted to datetime types so that time-based features (month, weekday, season, departure hour) could be derived.
- **Airport merge.** The flight logs were merged with the airport metadata on the departure airport identifier, attaching the geographic and airport-type attributes used later as features.

3.1 Order of Operations: Computing Propagation Before Filtering

The most important preprocessing decision concerns the order in which steps were performed. The previous-flight features, which describe whether an aircraft's immediately preceding flight was delayed, were computed on the full flight data before the dataset was filtered to the five target cities. Filtering to the five cities was then applied, so that the heavier later steps (weather download, encoding, and modeling) ran on the smaller subset.

The reason is that an aircraft's true previous flight may have departed from an airport that is not one of the five selected cities. If the previous-flight feature were computed after filtering, each aircraft's timeline would be fragmented and the feature would be weakened or wrong. Computing it first, on complete aircraft timelines, preserves the real sequence of flights for every aircraft and keeps the feature correct.

4. Feature Engineering

A large set of features was engineered and grouped by the kind of information it captures. The reasoning for each group is given below.

4.1 Date and Time Features

Year, month, day, weekday, quarter, a weekend indicator, season, departure hour, and a peak-hour indicator were derived from the flight date and the recorded departure time. These capture seasonal patterns and the daily cycle of travel, since congestion and the accumulation of small slippages build up during peak periods and busy hours.

4.2 Airport and Route Features

Route frequency, the number of flights from each origin city, a city traffic level (High, Medium, or Low), and counts of flights at each airport per day and per hour were computed. Busier airports and routes experience more congestion, which raises delay risk, so these features give the model a measure of how loaded the origin is at the time of a flight.

4.3 Weather Features

Fifteen weather-related features were added from the daily archive: mean, maximum, and minimum temperature; precipitation, rain, and snowfall totals; maximum wind speed and gusts; the raw weather code mapped into a condition group (Clear, Cloudy, Fog, Drizzle, Rain, Snow, Thunderstorm); and binary indicators for rainy, snowy, windy, and heavy-precipitation days together with an overall bad-weather indicator. Weather affects both flight safety and ground operations, and the binary indicators let the model separate a light shower from a genuinely disruptive event.

4.4 Operational Propagation Features

This group is the most important and received the most attention. It captures the fact that delays propagate through an aircraft's day and through chronically late routes:

- `prev_flight_delay_min`, `prev_flight_delayed`, `prev_same_day`, and `prev_delayed_same_day` describe whether the aircraft's previous flight was delayed and whether it occurred on the same day.
- `flight_seq_today`, `aircraft_delays_today`, and `aircraft_cum_delay_today` describe how the aircraft's day has gone so far: how many flights it has already operated and how much delay it has already accumulated.
- `mins_since_prev_flight` measures the turnaround time before the flight, since short turnarounds raise delay risk.

- `route_avg_delay` and `route_delay_rate` describe the historical delay behaviour of the specific route.

The justification for the whole group is that an aircraft delayed earlier in the day tends to remain delayed, and some routes are habitually late. These features act as proxies for the hidden operational causes—crew availability, maintenance, and congestion—that are not directly recorded in public data.

4.5 Leakage Prevention

Every operational feature was constructed to use only information available before the flight departs, which is essential for a model that would be used to predict delays in advance. The previous-flight and aircraft-daily-state features use only prior flights: the cumulative sums are shifted to exclude the current flight, so a flight never “sees” its own outcome. The route-history features `route_avg_delay` and `route_delay_rate` are computed as past-only expanding averages, so each flight sees only the route's earlier flights, never its own result.

In addition, columns that would leak the answer were explicitly excluded from the model: `delay_min` and `delay_category` (which define the target), `duration_min`, `first_seen`, and `last_seen` (actual post-flight values), and pure identifiers such as `id`, `icao24`, and the raw route text. Including any of these would produce an unrealistically high score that would collapse in real use, because the information would not be available at prediction time. Removing them is what keeps the reported performance honest.

5. Feature Selection

After encoding the categorical variables using one-hot encoding, the modeling dataset contained 52 features. A Random Forest model was then used to estimate feature importance, and features contributing less than 0.5% of the total importance were removed. As a result, 30 features were retained for all subsequent models.

Removing 22 low-importance features did not reduce the F1 score, indicating that these features contributed little to the prediction task. The reduced feature set simplified the models, decreased training time, and helped reduce the risk of overfitting.

Table 2: Feature counts before and after selection

Stage	Number of features
Encoded modeling matrix	52
Dropped (importance < 0.5%)	22
Retained for modeling	30

The composition of the dropped features is itself the clearest evidence for the project's central thesis. Almost every removed feature was a weather field or a coarse calendar or location indicator—for example the rainy, snowy, windy, and heavy-precipitation flags, the bad-weather indicator, snowfall totals, the weather-condition-group categories, the season indicators, the weekend and peak-hour flags, and several city and airport-type dummies. In other words, the Random Forest judged the weather features to be among the least useful, while the operational and schedule features were retained. This empirically supports the claim that operational factors outweigh weather, a point returned to in the discussion.

6. Modeling Methodology

The modeling procedure was designed to handle the class imbalance correctly and to compare the algorithms on equal terms.

- **Train/test split.** The data were split 80/20 with stratification on the target, which keeps the 4.99 percent delayed ratio identical in the training and test sets. Stratification is essential under strong imbalance, otherwise the rare class could be unevenly distributed between the two sets.
- **Feature scaling.** A StandardScaler was fitted on the training data and applied to both sets. Scaling is needed so that Logistic Regression converges well and K-Nearest Neighbors measures distances fairly; it is harmless for the tree-based models.
- **Class-imbalance handling.** The models that support it were trained with `class_weight` set to “balanced,” which forces the model to pay attention to the rare delayed class instead of ignoring it in favour of the majority.
- **Threshold tuning.** Rather than using the default 0.50 probability cutoff, each model was evaluated at the decision threshold that maximizes its F1 on the delayed class, because the default cutoff is rarely optimal under imbalance.
- **Sampling for comparison.** The algorithm comparison was run on a 300,000-row stratified sample for speed, since K-Nearest Neighbors in particular is impractically slow on millions of rows. Because the operational features were computed on the full data during preprocessing, sampling at the modeling stage does not weaken them.

7. Model Comparison and Results

A majority-class baseline and five classifiers were trained on the same selected features, each evaluated at its best F1 threshold. The results on the held-out test set are shown below, ordered by F1 on the delayed class

Table 3: Precision, recall, and F1 on the delayed class for all compared models

Model	Precision	Recall	F1
Random Forest	0.276	0.327	0.299
Decision Tree	0.248	0.279	0.263
K-Nearest Neighbors	0.232	0.299	0.261
Naïve Bayes (Gaussian)	0.244	0.228	0.236
Logistic Regression	0.207	0.266	0.233
Baseline (majority class)	0.050	1.000	0.095

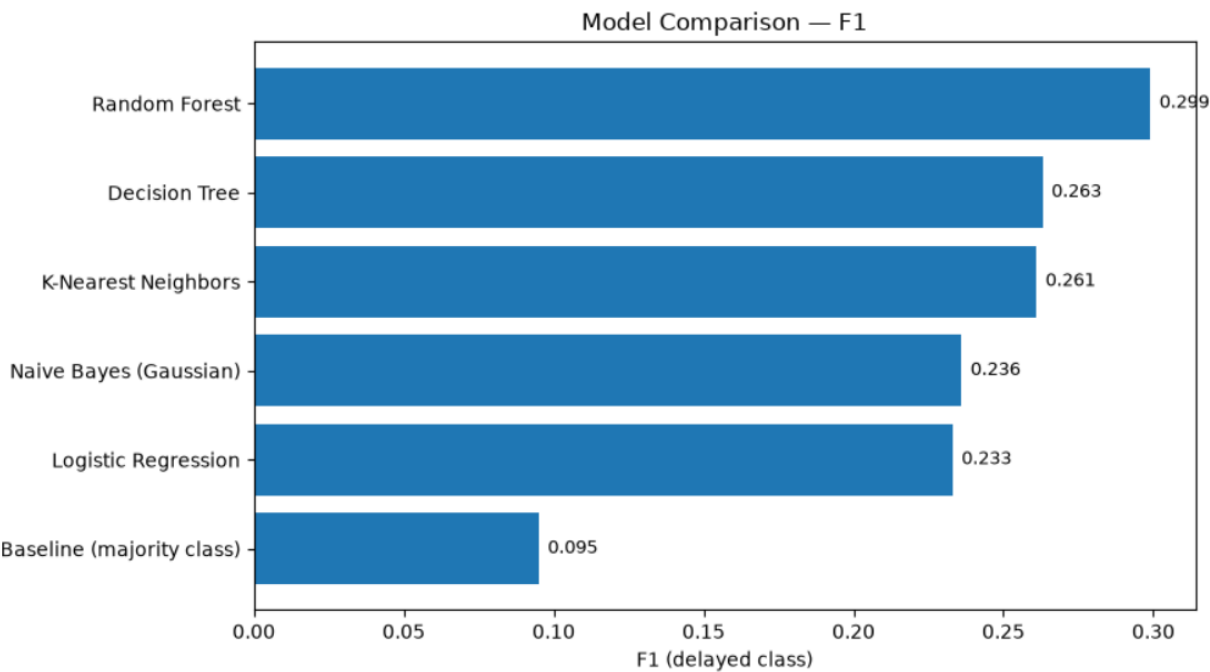


Figure 1: Model comparison by F1 on the delayed class

7.1 Interpretation

Random Forest achieved the highest F1 score among all evaluated models. As an ensemble of multiple decision trees, it can capture complex nonlinear relationships between operational, time, and weather features while reducing the instability of a single decision tree. For this reason, it was selected for further optimization.

The remaining models produced lower performance because of their underlying assumptions. Logistic Regression assumes a linear relationship between the input features and the target, which is often too simple for flight-delay prediction. Naive Bayes assumes that features are independent, while many of the operational and weather features in this dataset are correlated. K-Nearest Neighbors becomes less effective with high-dimensional data and also requires more computation time. Although a Decision Tree can model nonlinear patterns, it is more sensitive to variations in the training data, which makes Random Forest a more stable alternative.

The majority-class baseline further demonstrates why accuracy is not an appropriate evaluation metric for this problem. By predicting every flight as "not delayed," it achieved an accuracy of about 95%, but its F1 score was only 0.095 because it failed to identify delayed flights. This confirms that precision, recall, and F1 provide a much more meaningful evaluation for this highly imbalanced classification task.

8. Model Optimization and Enhancement

Three enhancements were applied to the Random Forest. Each is reported together with its result, including the one that did not help, since honestly reporting a failed experiment is part of a rigorous study.

8.1 Hyperparameter Tuning

A grid search optimizing F1 with three-fold cross-validation was run on a 50,000-row subsample for speed, then the best configuration was refitted on the full training set. The search selected a maximum depth of 25, a minimum of 10 samples per leaf, 200 trees, and the square-root feature rule. The tuned model improved the F1 from 0.299 to 0.312. The gain is real but small, which indicates that the original settings were already close to optimal; tuning validated the model rather than transforming it.

8.2 SMOTE Resampling

SMOTE generates synthetic examples of the minority class to balance the training set. It was tested because the related work reviewed in the earlier phase used resampling techniques, so it was reasonable to try here. The result was a decrease: F1 fell from 0.312 to 0.295. The synthetic delayed examples blurred an already weak class boundary instead of sharpening it. Finding that SMOTE did not help in this setting is a legitimate, honest result, not a failure of method.

8.3 Decision-Threshold Tuning

Because the default 0.50 cutoff is rarely optimal under imbalance, the probability threshold of the tuned Random Forest was scanned to find the value that maximizes F1. The best threshold was about 0.57, which traded a little recall for higher precision and produced the best F1 of 0.312. The trade-off curve is shown below.

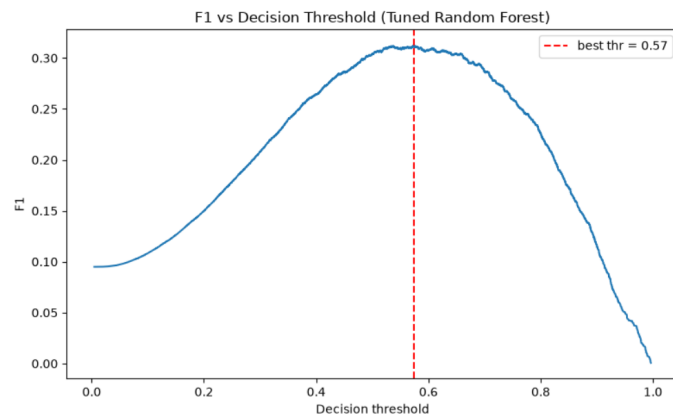


Figure 2: F1 as a function of the decision threshold for the tuned Random Forest; the best F1 occurs near a threshold of 0.57

8.4 The F1 Journey

Bringing the experiments together, the figure below traces F1 from the majority baseline through the five algorithms to the tuned Random Forest and the SMOTE variant. The progression rises from 0.095 at the baseline to 0.312 at the tuned model, then dips slightly with SMOTE.

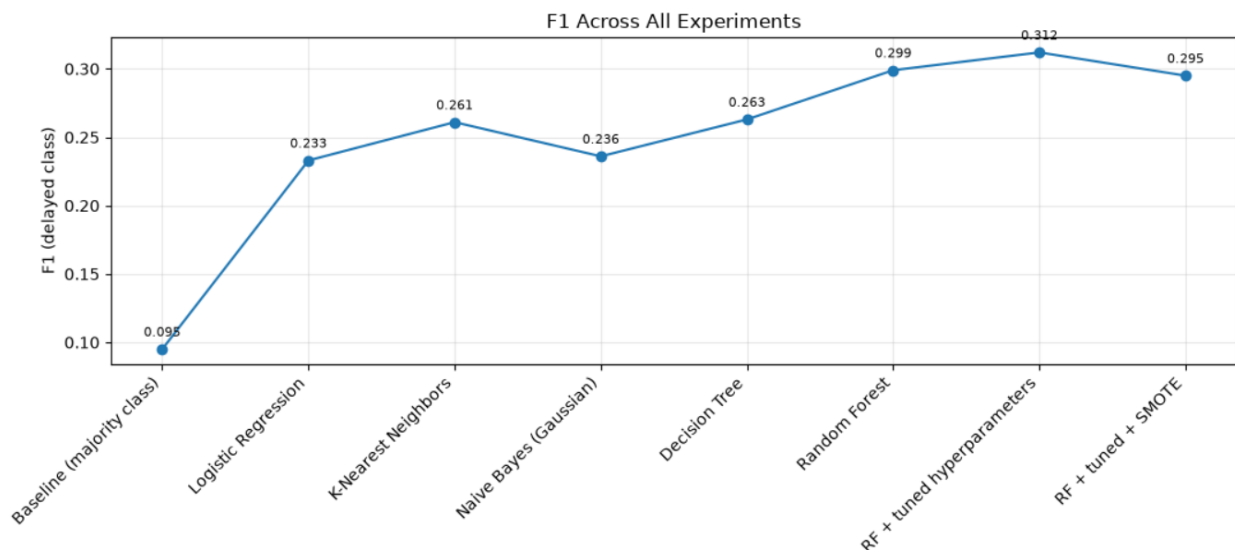


Figure 3: F1 across all experiments, from the majority baseline to the final tuned model

It is worth noting what did not happen. None of the enhancements produced a sudden leap to a much higher F1. Under a leakage-free methodology this is the expected and reassuring outcome: a suspicious jump to a very high score would have signalled that some feature was secretly revealing the answer. The modest, steady gains are evidence that the operational and route-history features were computed correctly, using only information available before each flight.

9. Final Model and Evaluation

The final model is the tuned Random Forest evaluated at the F1-optimal threshold of about 0.57. On the held-out test set it achieves an F1 of approximately 0.31 on the delayed class, with precision near 0.29 and recall near 0.34. The classification report and confusion matrix are interpreted below.

Table 4: Classification report of the final tuned Random Forest on the test set

Class	Precision	Recall	F1
Not Delayed	0.97	0.96	0.96
Delayed	0.29	0.34	0.31
Accuracy (overall)			0.93

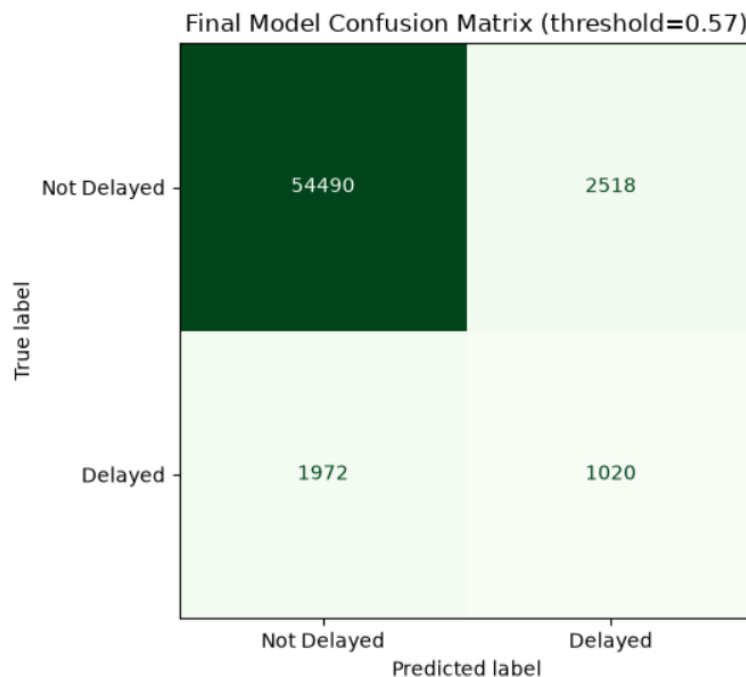


Figure 4: Confusion matrix of the final model at a decision threshold of 0.57

Reading these results in plain terms: of the flights that were truly delayed, the model correctly catches about 34 percent (recall), and when it flags a flight as likely to be delayed it is correct about 29 percent of the time (precision). The overall accuracy of roughly 93 percent looks high only because the negative class dominates, and it should be set aside in favour of the delayed-class metrics.

The most useful way to express what the model has learned is in relation to the base rate. When the model flags a flight as likely delayed, that flight is delayed about six times more often than a randomly chosen flight (around 29 percent versus the 4.99 percent base rate). The model has therefore learned real, useful signal rather than guessing, even though its absolute F1 is modest.

10. Discussion

The experimental results lead to three main observations.

First, operational factors appear to have a greater impact on flight delays than weather conditions. This conclusion is supported by the feature-selection results, where most weather-related and calendar features were assigned low importance, while operational and schedule-related features remained among the strongest predictors. In particular, features describing delay propagation and route history contributed more to the final predictions than weather variables.

The second observation is that all evaluated models achieved very similar performance. Although different algorithms and optimization techniques were tested, the F1 score remained close to 0.30 for the delayed class. This suggests that the main limitation is related to the available data rather than the choice of machine-learning algorithm.

Finally, many important factors that influence flight delays are not included in public datasets. Information such as crew availability, aircraft maintenance, gate congestion, and air traffic control restrictions is unavailable, even though these factors play a major role in real-world delays. As a result, the models have limited information to learn from, which naturally restricts their prediction performance. This finding is also consistent with the observations from the previous phase, where the relationship between weather and delays varied across different cities and seasons, showing that flight delays depend on multiple interacting factors rather than weather alone.

11. Limitations

- Weather is represented by daily summaries rather than hourly observations at the actual departure time, so short-lived conditions that cause real delays are averaged away.
- Weather was collected only at departure airports, not at arrival airports or along the flight path, so a large part of the atmospheric influence on a flight is absent.
- The most important operational variables, including crew availability, maintenance, air-traffic-control restrictions, and gate congestion, are not included in the dataset. This is the biggest limitation of the study and the main reason for the performance ceiling.
- The aircraft-daily-state features were computed on the five-city subset, so flights operated from airports outside the selected cities are not included in those daily statistics. Computing these features on the complete dataset would provide a more accurate representation of aircraft operations.
- The study focuses on flights from five European cities, so the findings may not generalize to other regions or to the global aviation network.
- The dataset is highly imbalanced, with only a small percentage of flights classified as delayed. Although class weighting and threshold tuning were applied, the imbalance still makes it difficult for the models to identify delayed flights accurately.
- Some potentially useful information, such as airline operational policies, aircraft characteristics, and airport resource availability, was not available in the public datasets and therefore could not be included in the prediction models.
- The models were trained and evaluated using historical data from a specific time period. Future changes in airline operations, airport management, or travel patterns may reduce the models' performance when applied to newer data.

12. Conclusion and Future Work

This phase built and compared a set of classifiers to predict whether a flight is delayed by more than fifteen minutes. The Random Forest performed best and, after hyperparameter and threshold tuning, reached an F1 of about 0.31 on the delayed class. Operational features describing delay propagation and route history outweighed weather features, and every algorithm and enhancement converged on a similar modest ceiling. That convergence is the real result: it demonstrates the limit of what public flight and weather data can reveal about delays, because the dominant causal variables are not recorded.

Future work should focus on closing that data gap rather than on more complex models. The most valuable additions would be operational data—crew rosters, maintenance logs, and air-traffic-control information—together with arrival-airport and en-route weather. The aircraft-propagation features should be computed on the full unfiltered data during preprocessing to remove the approximation noted above. Finally, reframing the target at an airport-day level of aggregation may yield a more predictable quantity than individual-flight occurrence, offering a complementary view of the same problem.

References

Hugging Face. Flight Delays Europe 2023–2025 Dataset. Available at:

<https://huggingface.co/datasets/345rf4gt56t4r3e3/flight-delays-europe-2023-2025>

OurAirports. Airports Database. Available at:

<https://ourairports.com/data/>

Open-Meteo. Historical Weather API Documentation. Available at:

<https://open-meteo.com/>

Scikit-learn Developers. Scikit-learn Documentation. Available at: <https://scikit-learn.org/>

Imbalanced-learn Developers. Imbalanced-learn Documentation (SMOTE). Available at:

<https://imbalanced-learn.org/>

Pandas Development Team. Pandas Documentation. Available at:

<https://pandas.pydata.org/>

NumPy Developers. NumPy Documentation. Available at:

<https://numpy.org/>

Matplotlib Development Team. Matplotlib Documentation. Available at:

<https://matplotlib.org/>